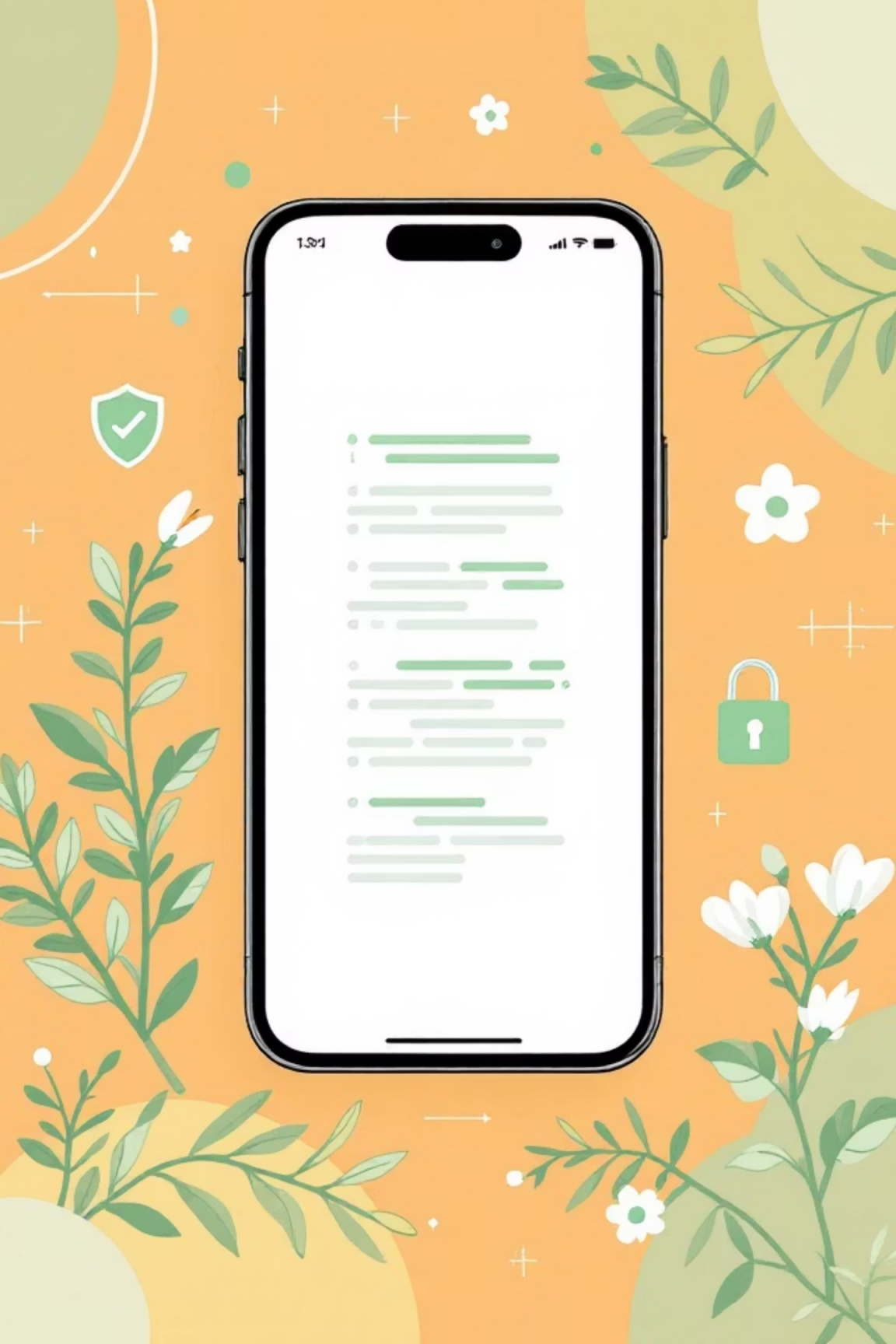




Obfuscating Dart Code in Flutter

Protect your Flutter app from reverse engineering while maintaining debuggability

What is Code Obfuscation?

Normal Build

APK can be reverse engineered

- Readable variable names
- Clear logic flow
- Visible API calls

Obfuscated Build

Code becomes unreadable

- Variables renamed (a, b, c)
- Logic obscured
- API endpoints hidden

Obfuscation transforms your human-readable Dart code into a format that's nearly impossible for attackers to understand—protecting your intellectual property and sensitive logic.

Real-Life Analogy

Normal Code 📖

Like an open book where anyone can read your story, understand your characters, and follow the plot from beginning to end.

Obfuscated Code 🗝️

Like a secret code that only you can decipher. The story exists, but the meaning is hidden behind layers of encryption and transformation.



Before and After Obfuscation

✗ Without Obfuscation

```
int userBalance = 5000;  
String apiKey = "my_secret_key";  
String token = generateAuthToken();  
  
void fetchUserData() {  
    // Clear logic  
    // Easy to understand  
}
```

Hackers can easily understand your code structure, identify sensitive data, and exploit vulnerabilities.

✓ With Obfuscation

```
int a = 5000;  
String b = "my_secret_key";  
String c = d();  
  
void e() {  
    // What does this do?  
    // Impossible to tell  
}
```

Meaning is hidden through systematic renaming of variables, functions, and classes—making reverse engineering significantly more difficult.

Understanding the Command

01

flutter build apk

Builds your Android application package (APK file) for distribution

02

--obfuscate

Transforms readable Dart code into unreadable format by renaming variables, functions, and class names

03

--split-debug-info=

Saves original code mapping to decode crash logs later—this is the critical piece most developers forget

Why split-debug-info is Critical

1

Obfuscation Happens

Code gets transformed into unreadable format

2

Crash Occurs

Error log shows obfuscated class names and methods

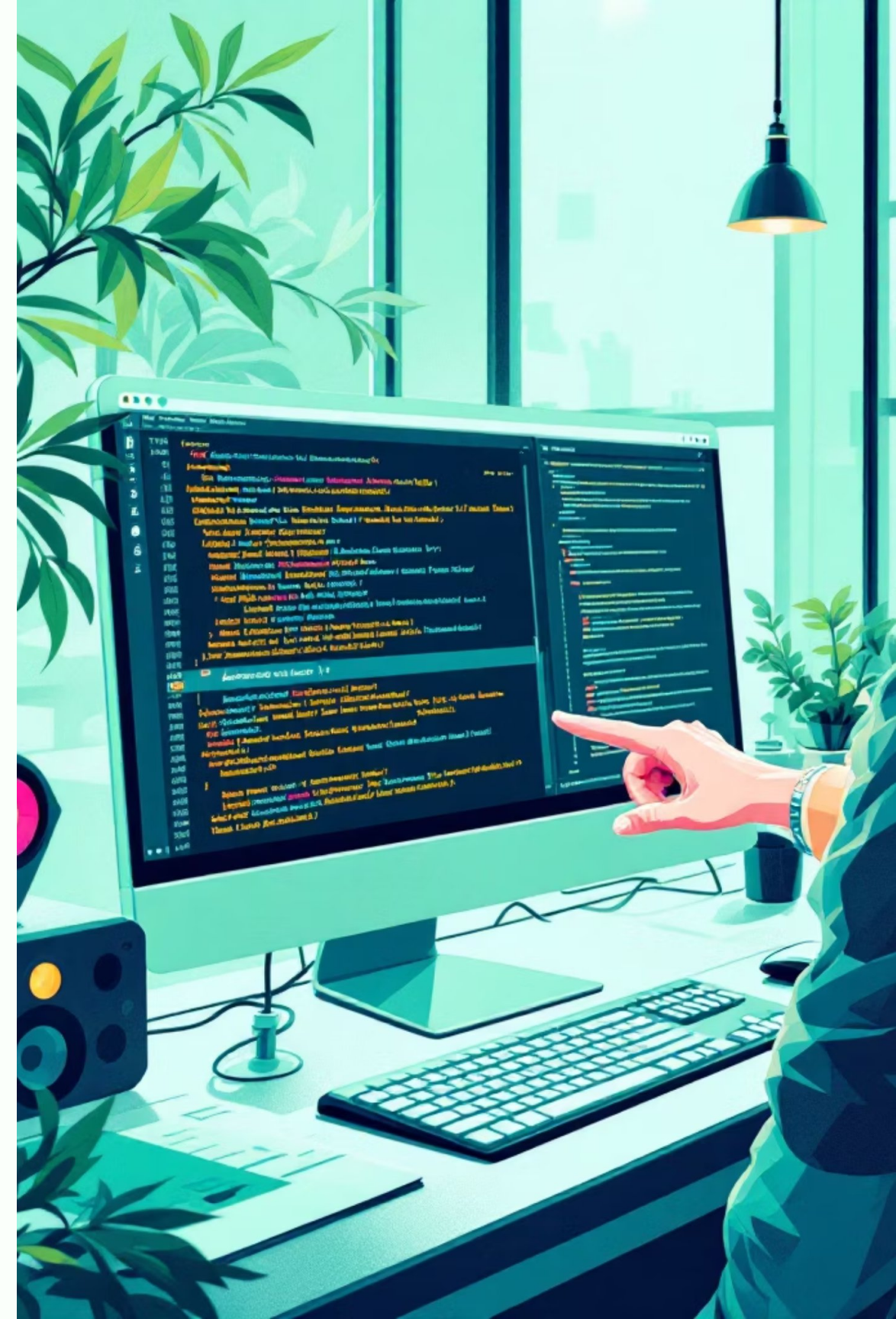
3

Mapping File Used

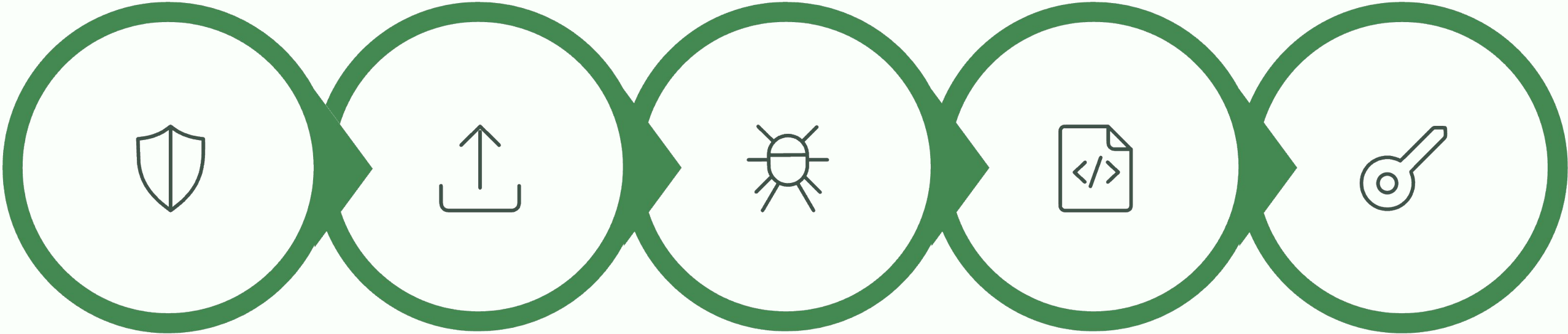
Decode to original readable code structure

❏ **Without mapping:** "Error in class A at method B" is useless

With mapping: "Error in UserController at fetchUserData" is actionable



Complete Build Flow



**Obfuscate
Build**

**Distribute
App**

**Production
Crash**

**Unreadable
Log**

**Apply
Mapping**

Example Usage

Command Syntax

```
flutter build apk \  
  --obfuscate \  
  --split-debug-info=debug-info/
```

This creates a `debug-info/` folder containing your mapping files, such as `app.android-arm64.symbols`

Folder Structure

```
debug-info/  
├── app.android-arm64.symbols  
├── app.android-arm.symbols  
└── app.android-x64.symbols
```

Each architecture variant gets its own mapping file for comprehensive crash log decoding.

Good vs Bad Practices

✗ Never Do This

```
flutter build apk --obfuscate
```

Problem: No debug info saved—you can't decode crashes later

✓ Always Do This

```
flutter build apk \  
  --obfuscate \  
  --split-debug-info=debug-info/
```

Result: Secure app that remains debuggable

Important Best Practices

1

Never lose debug-info folder

If lost, you can NEVER decode crash logs again. Treat it like source code.

2

Keep debug-info secure

Do NOT upload with APK. Store locally or in secure cloud storage with restricted access.

3

Use in production only

Don't enable obfuscation during development—it makes debugging unnecessarily difficult.

4

Works with both APK and App Bundle

Recommended: Use app bundle for Google Play distribution

